PawTracker

Dog Sport Tracker App

Mobile Application Development Project

Farnaz Jamilpanah & Emmi Korhonen

Agenda

01	Project Overview
02	Design
03	Tech Stack & Architecture
04	Tracking (Map & GPS)
05	Demo
06	Development Process

01 Project Overview

Introduction

Problem

Dog owners often don't know if their pets are getting enough physical activity → Might affect negatively to dog's health.

It's hard to track walking routes, distances and durations manually.

Solution

PawTracker Dog Sport Tracker App provides a simple, visual interface that shows dog's walking routes.

Core Mission

PawTracker helps owners track their dog's activity, identify if their pets are getting enough exercise, and improve their overall wellbeing.

Target Users

- ***Active & Conscious Dog Owners***
- ***Tech Savvy Users***

Core Features

The app tracks walking routes via GPS, calculating distance and duration, and stores walk data for long term tracking.

- **GPS tracking:** Records the pet's walking route in real time
- **Map view:** Shows the route visually on a map
- **Activity statistics:** Distance, duration and daily totals
- **History:** Stores past walks using Room database
- **Dog profile:** Stores dog information
- **Light and Dark Theme**

Scrum & Team Roles

Scrum

Approach

Project duration 18.3-6.5.2026

- Sprint length: 1 week
- Total: 6 sprints

Communication

- Daily Scrum on Mon, Wed, Fri
- Ongoing discussion on Discord

Tracking

- Product backlog, Sprint backlog

Tasks

- **Priority:** Low, Medium, High
- **Story Points Scale:** 1–6

Team Roles

Product Owner: Both

Developers: Cross-functional collaboration

Scrum Master:

Planning Sprint – Emmi

Sprint 1 – Farnaz

Sprint 2 – Emmi

Sprint 3 – Farnaz

Sprint 4 – Emmi

Sprint 5 – Farnaz

Sprint 6 – Emmi

Product Backlog

As a new user, I want to create a dog profile so I can track my pet's activity.

As a user, I want to view and set my dogs profile breed, weight and age.

As a user, I want to set daily walk goals so I can stay consistent.

As a user, I want a "Start Walk" button so I can begin tracking easily.

As a dog owner, I want the app to track my walk using GPS so that I can record my dog's distance, pace, and route accurately.

As a user, I want to see a live map of my walk route so that I can recognize the runs by area.

As a user, I want to see recent walks with distance and time so I stay motivated.

As a user, I want to filter history by week or day.

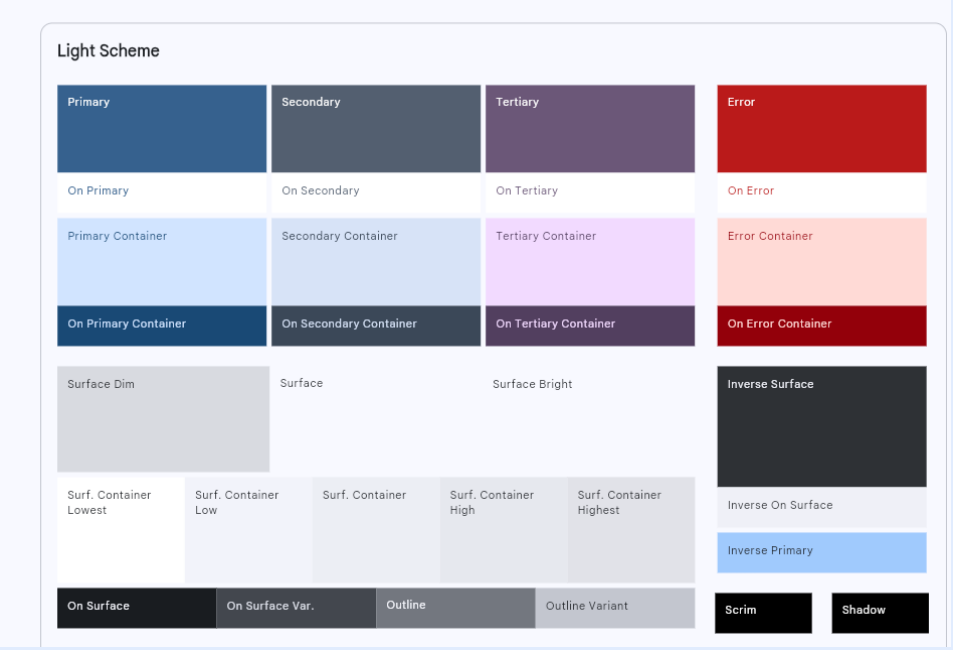
As a user, I want to compare runs to recommended activity, so that I know If my dog is getting enough exercise.

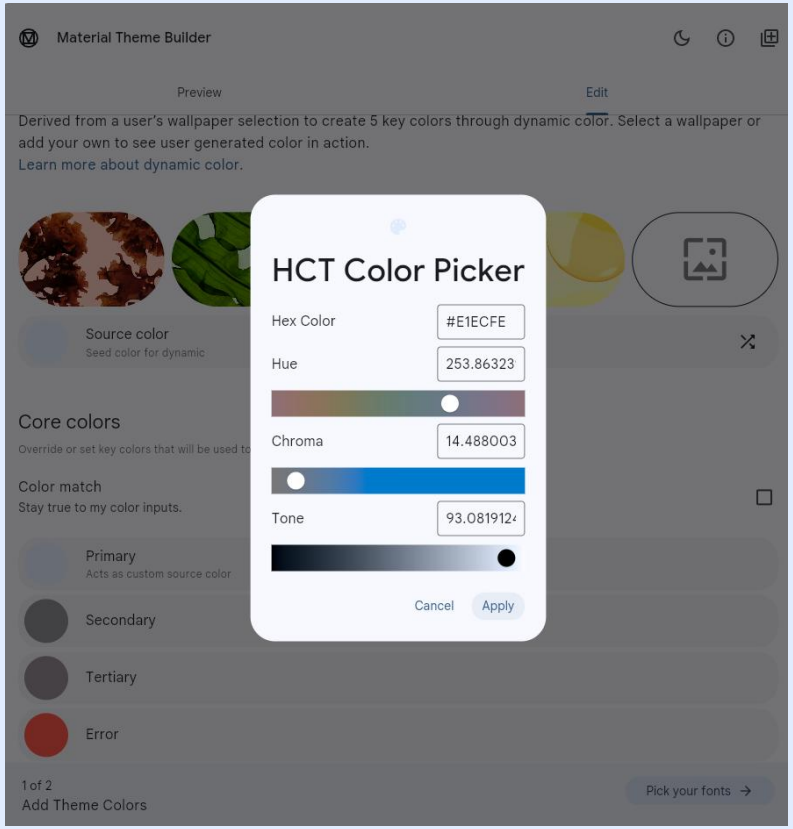
As a user, I want to edit, add and delete my dog's info anytime.

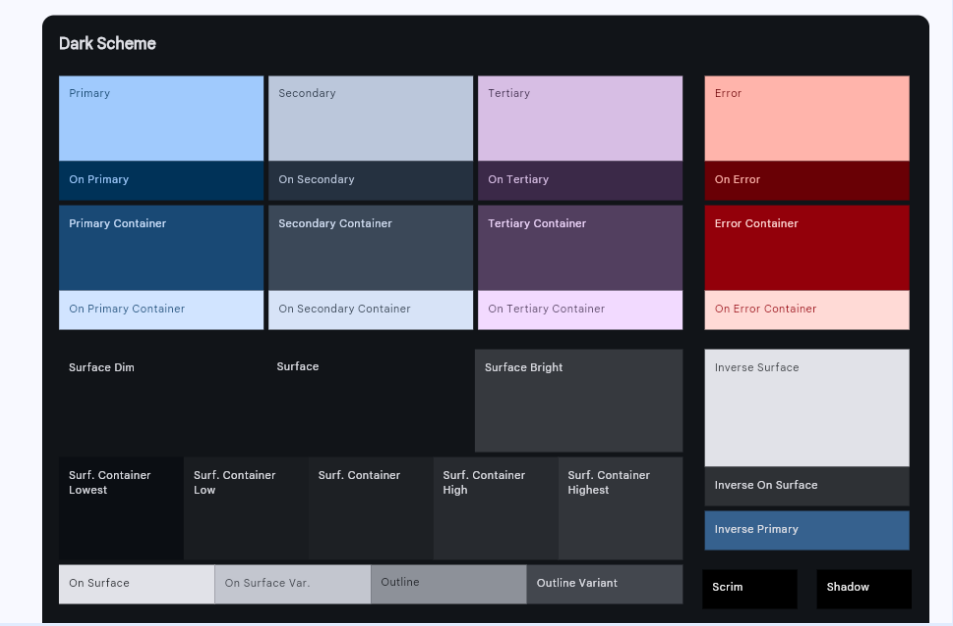
02 Design

Theme

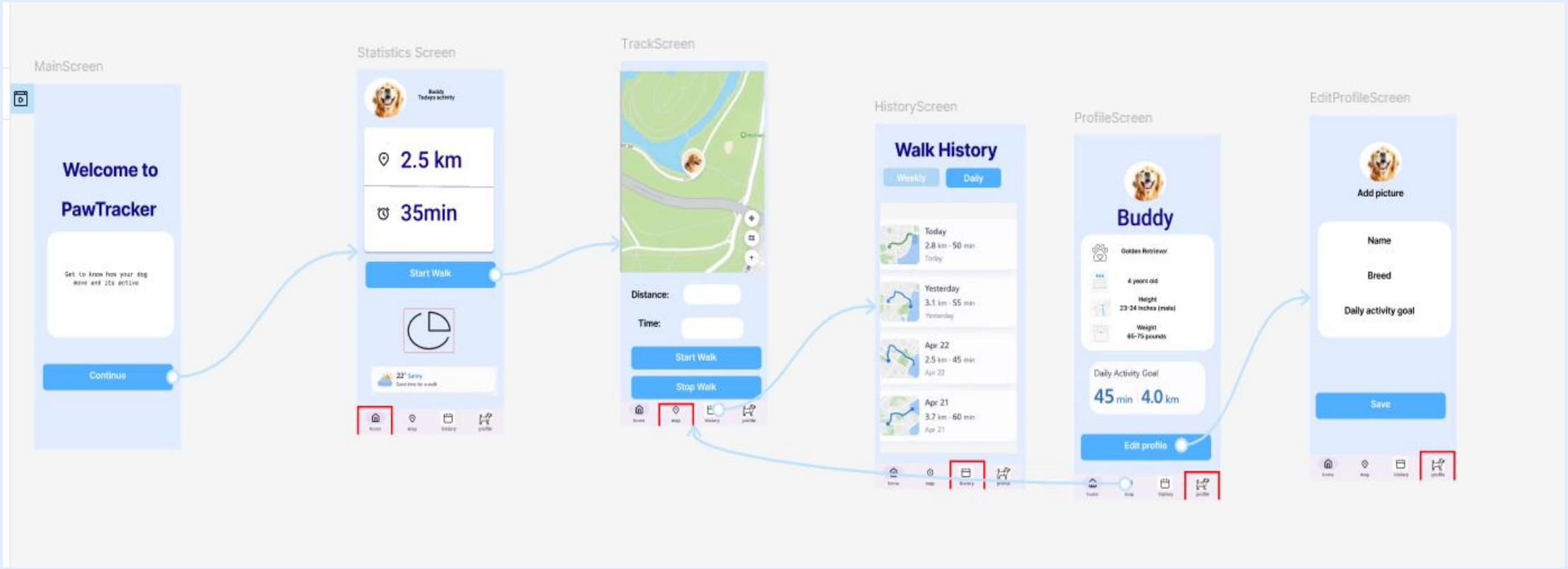
- **Material Design 3**
- **Theme, Color, Shape & Spacing**







Figma


MainScreen
Welcome to
PawTracker
Get to know how your dog move and its active
Continue
Statistics Screen
2.5 km
35min
Start Walk
TrackScreen
Distance:
Time:
Start Walk
Stop Walk
HistoryScreen
Walk History
Weekly
Daily
Today
Yesterday
Apr 22
Apr 21
ProfileScreen
Buddy
Golden Retriever
4 years old
Daily Activity Goal
45 min 4.0 km
Edit profile
EditProfileScreen
Add picture
Name
Breed
Daily activity goal
Save

03 Tech Stack & Architecture

Technology Stack

Core & Architecture

- Language: **Kotlin**
- UI Framework: **Jetpack Compose & Navigation Component**
- Architecture: **MVVM (ViewModel + StateFlow)**
- Dependency Management: **Custom Factory**
- Testing: **Junit (Unit tests), UI tests, database tests**

Data & Persistence

- Database: **Room**
- Preferences: **Jetpack DataStore**

Hardware & APIs

- Extra Hardware: **GPS & Location Services** (Fused Location Provider)
- APIs: **Google Maps SDK**

Tools

- IDE: **Android Studio**
- Version Control: **GitHub**
- Project Management: **Jira**
- Design: **Figma, Adobe Firefly**
- Communication: **Discord**

Application Architecture

```
com.example.pawtracker
│
├── data/                                # Data layer: All data fetching and persistence
│   ├── local/                           # Local data sources (Room & DataStore)
│   │   ├── converters/                  # Room TypeConverters ( LocationPointConverter)
│   │   ├── preferences/                 # DataStore for onBoarding (PreferenceRepository)
│   │   ├── AppDatabase.kt               # Room database definition
│   │   ├── DogProfileDao.kt             # Data Access objects for Room
│   │   ├── ...                          #
│   │   ├── DogProfileEntity.kt          # Database entities tables
│   │   ├── ...                          #
│   ├── repository/                      # Repositories: Single source of truth for the app
│   │   ├── DogProfileRepository.kt      # Repository interfaces
│   │   ├── ...                          #
│   │   ├── DogProfileRepositoryImpl.kt  # Repository implementations
│   │   ├── ...                          #
│   ├── mapper/                          # Data mappers
│   │   └── WalkMappers.kt               #
│   └── model/                           #
│       └── LocationPoint.kt             #
│
├── ui/                                  # UI layer: All Compose-related
│   ├── components/                      # UI components (NavBar, NavigationRail)
│   ├── navigation/                      # Navigation logic (NavGraph, Screens, NavigationType)
│   ├── theme/                           # App styling (Color, Shape, Spacing, Theme, Type)
│   ├── editprofile/                     # EditProfile deature files (Screen, ViewModel, UI State)
│   │   ├── EditProfileScreen.kt
│   │   ├── EditProfileUiState.kt
│   │   └── EditProfileViewModel.kt
│   ├── history/                         # History feature files
│   ├── main/                            # Main feature files
│   ├── profile/                         # Profile feature files
│   ├── statistics/                      # Statistics feature files
│   └── tracking/                        # Tracking feature files
│
├── utils/                               # Shared utility classes
│   ├── Time.kt
│   └── ViewModelFactory.kt              # Custom Factory for ViewModel injection
│
├── MainActivity.kt                      # Entry point for the application
└── PawTrackerApplication.kt             # Application class for global initialization
```

Testing

```
Terminal   Local ×   Git Bash ×

PS C:\Users\Fari\AndroidStudioProjects\mobile-app-project\pawtracker> ./gradlew jacocoFullReport

> Configure project :app
WARNING: The option setting 'android.disallowKotlinSourceSets=false' is experimental.
The current default is 'true'.
Starting 17 tests on Medium_Phone(AVD) - 14
Medium_Phone(AVD) - 14 Tests 1/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 2/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 3/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 4/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 6/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 8/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 10/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 12/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 13/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 14/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 15/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 16/17 completed. (0 skipped) (0 failed)
Medium_Phone(AVD) - 14 Tests 17/17 completed. (0 skipped) (0 failed)
Finished 17 tests on Medium_Phone(AVD) - 14

BUILD SUCCESSFUL in 5m 14s
77 actionable tasks: 9 executed, 68 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.2.1/userguide/configuration_cache_enabling.html

PS C:\Users\Fari\AndroidStudioProjects\mobile-app-project\pawtracker>
```

localhost:63342/Paw%20Tracker/build/reports/jacoco/jacocoFullReport/html/index.html?_ijt=t5q74u...

Whatsapp PC - Chr... | Gmail | YouTube | Maps | Stored Billing | Ticketmaster - My A...

app — Sessions

app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.example.pawtracker.ui.editprofile		0%		0%	121	121	243	243	41	41	6	6
com.example.pawtracker.ui.tracking		32%		18%	128	146	199	283	29	38	7	10
com.example.pawtracker.ui.history		0%		0%	105	105	173	173	32	32	7	7
com.example.pawtracker.ui.profile		65%		35%	156	185	76	285	6	34	0	6
com.example.pawtracker.ui.navigation		5%		0%	60	68	79	86	22	30	2	9
com.example.pawtracker.ui.statistics		69%		37%	113	138	37	213	5	29	0	7
com.example.pawtracker.ui.main		0%		0%	38	38	69	69	16	16	7	7
com.example.pawtracker.data.repository		0%		0%	37	37	80	80	25	25	4	4
com.example.pawtracker.ui.theme		95%		50%	19	98	0	190	4	79	0	6
Total	10,046 of 16,050	37%	976 of 1,222	20%	777	936	956	1,622	180	324	33	62

Created with JaCoCo 0.8.10.202304240956

04 Tracking (Map & GPS)

Tracking

1. When the map screen is opened, the last location is loaded and displayed

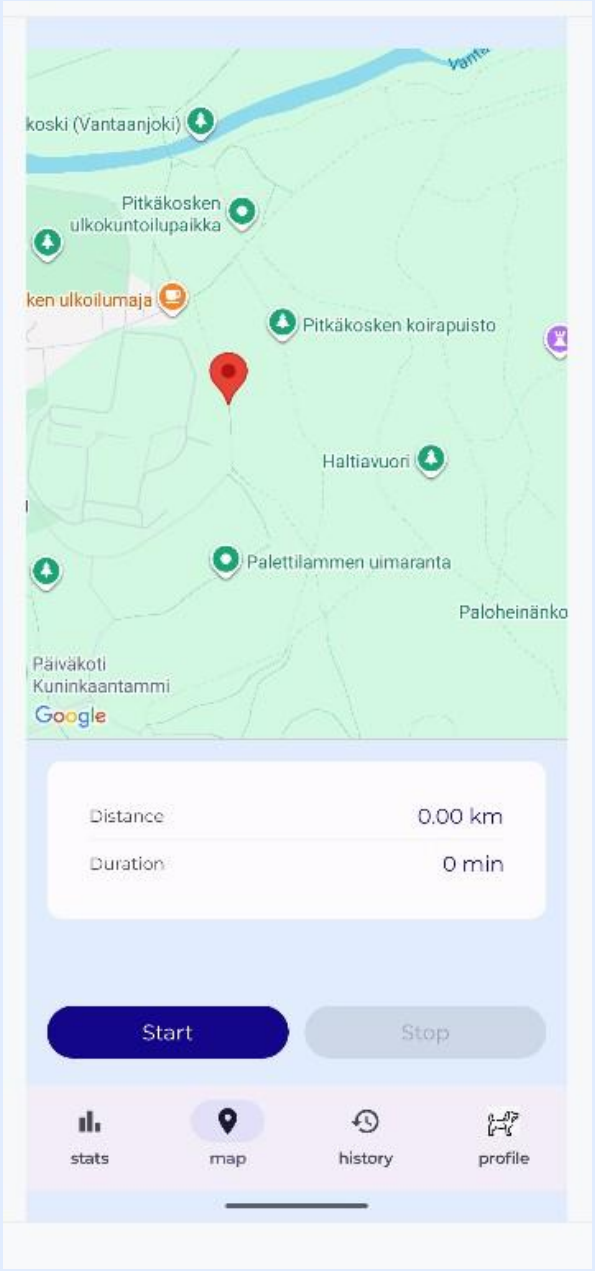
2. Start tracking initiates continuous GPS updates from repository

3. Incoming location points are handled in the ViewModel and appended to UI state

4. UI observes this state and updates polylane and tracking statistics automatically

5. Distance is computed incrementally using GPS points and duration is derived from system time

6. Once tracking stops, the walk is saved to Room database



1. Screen Initialization: When the screen is opened, the TrackingViewModel initializes and calls loadLastLocation to fetch the user's last known location.

```
init {
    loadLastLocation()
}
```

2. The GPSRepository provides the last known location via loadLastLocation()

```
1 Usage
fun loadLastLocation() {
    if (!useMockLocation) {
        gpsRepository.getLastLocation { point ->
            if (point != null) {
                _uiState.update { it.copy(currentLocation = point) }
            }
        }
    }
}
```

Once received, the ViewModel updates uiState.currentLocation

3. The TrackingMap observes state changes. When currentLocation updates, the camera moves using animate

```
LaunchedEffect( key1 = uiState.currentLocation) {
    uiState.currentLocation?.let { point ->
        val target = LatLng(point.latitude, point.longitude)
        cameraPositionState.animate(
            update = CameraUpdateFactory.newLatLngZoom( latLng = target,  zoom = 15f),
            durationMs = 1000
        )
    }
}
```

4. Start Tracking: Tracking begins when user presses the Start button

```
onStart = { viewModel.startTracking() },
```

5. Tracking state is enabled and coroutine (trackingJob) **starts collecting location updates.**

Real GPS data comes from gpsRepository.startLocationUpdates.

```
1 Usage
fun startTracking() {
    _uiState.value = TrackingUiState(tracking = true)

    startTime = System.currentTimeMillis()
    lastPoint = null

    trackingJob?.cancel()
    trackingJob = viewModelScope.launch {

        if (useMockLocation) {
            mockLocationFlow().collect { point ->
                handleNewPoint(point)
            }
        } else {
            gpsRepository.startLocationUpdates { point ->
                handleNewPoint(point)
            }

        }
    }
}
```

6. Location updates from repository

```
@RequiresPermission(anyOf = [
    Manifest.permission.ACCESS_FINE_LOCATION,
    Manifest.permission.ACCESS_COARSE_LOCATION
])
override fun startLocationUpdates(onUpdate: (LocationPoint) -> Unit) {

    if (!hasLocationPermission()) return

    setupLocationRequest()

    locationCallback = object : LocationCallback() {
        override fun onLocationResult(locationResult: LocationResult) {
            val loc: Location = locationResult.lastLocation ?: return
            onUpdate(
                LocationPoint(
                    latitude = loc.latitude,
                    longitude = loc.longitude,
                    time = loc.time
                )
            )
        }
    }

    fusedLocationProviderClient.requestLocationUpdates(
        p0 = locationRequest,
        p1 = locationCallback,
        p2 = Looper.getMainLooper()
    )
```

7. Handling New GPS Points: Each incoming point is processed in handleNewPoint:

```
2 Usages
private fun handleNewPoint(point: LocationPoint) {

    if (lastPoint != null) {

        val distanceKm = calculateDistance( a = lastPoint!!,  b = point)
        val timeDiffMs = point.time - lastPoint!!.time

        val timeDiffSec = if (timeDiffMs <= 0) 1.0 else timeDiffMs / 1000.0

        val speed = distanceKm / (timeDiffSec / 3600.0) // km/h
        // dog cannot go > 20 km/h
        // Ignore impossible speeds
        if (speed > 25) return

        // Ignore tiny GPS noise (< 1 meter)
        if (distanceKm < 0.001) return
    }
```

8. Filtering and Calculations:

Invalid data is filtered:

- Ignores unrealistic speeds (> 25 km/h) and
- movements under 1 meter

Calculations:

- Distance is accumulated incrementally
- Duration is based on system time:

```
distance = state.distance + addedDistance,
```

```
startTime = System.currentTimeMillis()
```

```
val elapsedTime = System.currentTimeMillis() - startTime
```

9. State Updates: The ViewModel updates the state. This triggers the automatic UI recomposition (Stateflow)

The map polyline grows in real time. Distance and time statistics update continuously.

```
11          _uiState.update { state ->
12              state.copy(
13                  currentLocation = point,
14                  points = state.points + point,
15                  distance = state.distance + addedDistance,
16                  time = elapsedTime
17              )
18          }
19      }
```

9. Rendering the route: The polyline grows dynamically as new points aare added.

```
94      Polyline(
95          points = uiState.points.map {
96              LatLng(it.latitude, it.longitude)
97          }
98      )
99
100     uiState.currentLocation?.let { point ->
101         Marker(
102             state = MarkerState(
103                 position = LatLng(point.latitude, point.longitude)
104             ),
105             title = "Current"
106         )
```

10. Stoping Tracking and Saving Data: When the user stops tracking, walkEntity is created and walk and points are saved to Room database.

```
1 Usage
fun stopTracking() {
    _uiState.update { it.copy(tracking = false) }

    trackingJob?.cancel()
    trackingJob = null

    if (!useMockLocation) {
        gpsRepository.stopLocationUpdates()
    }

    val endTime = System.currentTimeMillis()

    val points = _uiState.value.points

    val walk = WalkEntity(
        startTime = startTime,
        endTime = endTime,
        distance = (_uiState.value.distance * 1000).toFloat(),
        duration = endTime - startTime,
        pointCount = points.size,
        previewPolyline = null
    )

    viewModelScope.launch {
        walkRepository.insertWalkWithPoints(
            walk = walk,
            points = points
        )
    }
```

Future Improvement:
Currently GPS points stored in memory during tracking and saved at the end. Potential improvement is to add points to database in real time.

walks

gps_points | walks

Live updates

50

	id	startTime	endTime	distance	duration	pointCount	previewPolyline
1	1	1777453594486	1777453697519	235.4129180908203	103033	14	NULL
2	2	1777455786906	1777455890692	202.57644653320312	103786	14	NULL
3	3	1777747311049	1777747383543	259.0145568847656	72494	9	NULL
4	4	1777804742308	1777805069985	1195.1910400390625	327677	43	NULL
5	5	1777841793541	1777841854979	0.0	61438	1	NULL
6	6	1777841856570	1777841959250	79.8366470336914	102680	3	NULL
7	7	1777841982981	1777842013276	28.621496200561523	30295	3	NULL
8	8	1777990134357	1777990201074	133.90789794921875	66717	9	NULL
9	9	1777990236731	1777990254556	0.0	17825	1	NULL
10	10	1777990261510	1777990378805	0.0	117295	1	NULL
11	11	1777990380642	1777990433953	0.0	53311	1	NULL
12	12	1777990436324	1777990717388	403.0207214355469	281064	39	NULL
13	13	1777994399696	1777994440258	70.3653793334961	40562	5	NULL
14	14	1777995597824	1777995642997	89.17387390136719	45173	6	NULL
15	15	1777997972973	1777998138398	379.7782287597656	165425	23	NULL

gps_points

gps_points

Live updates

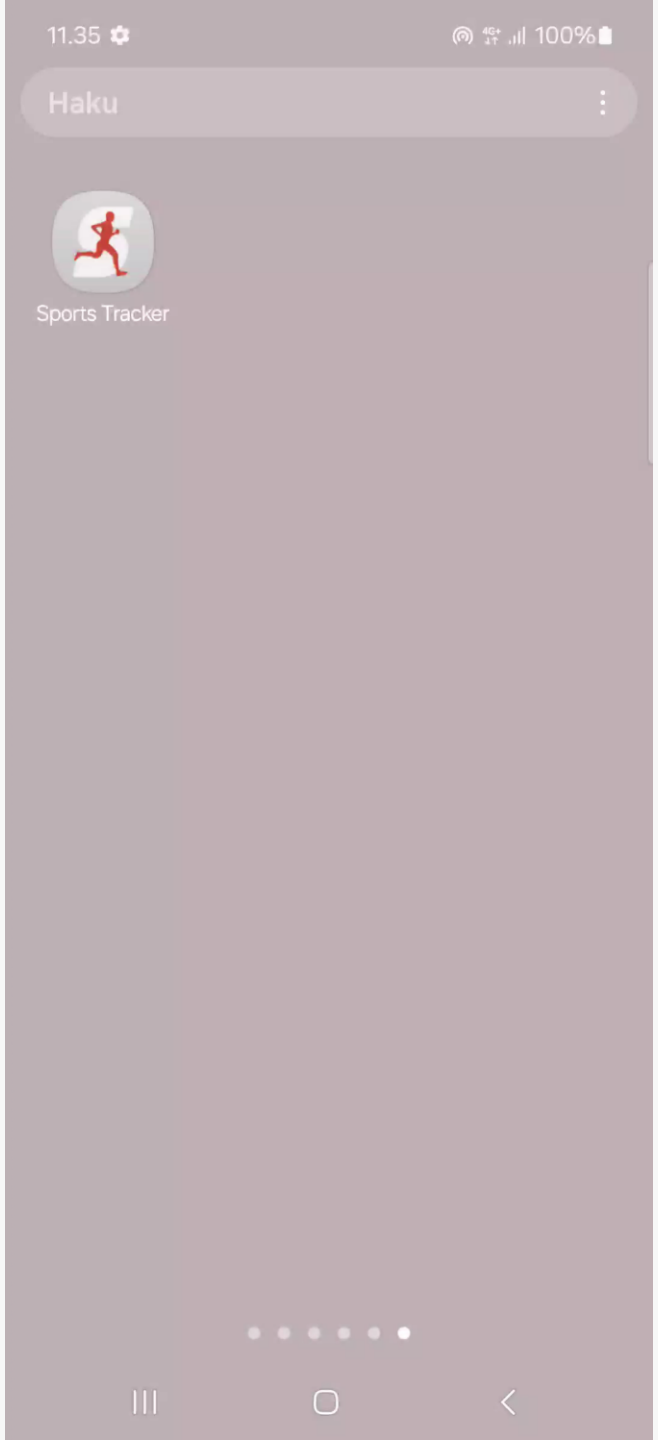
50

	id	walkId	sequence	timestamp	latitude	longitude
1	1	1	0	1777453602206	60.1766988	24.9336879
2	2	1	1	1777453609721	60.1764858	24.9338201
3	3	1	2	1777453615845	60.1762666	24.9339505
4	4	1	3	1777453623101	60.1760497	24.9340807
5	5	1	4	1777453630164	60.1758249	24.9342351
6	6	1	5	1777453638609	60.175629	24.934372
7	7	1	6	1777453646747	60.1755876	24.9345491
8	8	1	7	1777453653240	60.1754615	24.9346179
9	9	1	8	1777453660464	60.1753362	24.9346832
10	10	1	9	1777453666787	60.1752433	24.9347436
11	11	1	10	1777453674228	60.1751033	24.934837
12	12	1	11	1777453681771	60.1749732	24.9349051
13	13	1	12	1777453688797	60.1748661	24.9349603
14	14	1	13	1777453696064	60.1747251	24.935042
15	15	2	0	1777455794710	60.1762347	24.9339701
16	16	2	1	1777455800869	60.1760461	24.9340832
17	17	2	2	1777455808123	60.1758267	24.9342339
18	18	2	3	1777455815008	60.1756673	24.9343623
19	19	2	4	1777455822220	60.1755873	24.9344809

com > example > pawtracker > ui > tracking > TrackingViewModel > _uiState

34:63 CRLF UTF-8

05 Demo


11.35
100%
Haku
Sports Tracker

06 Development Process

Timeline

	Dates	Focus	Tasks
Sprint 0	18.3.-25.3	Concept	Project Idea, Project Plan, MVP definition, Product Backlog, Figma Design
Sprint 1	25.3-1.4	Core Foundation	GPS Tracking, Google Maps API, Tracking view, Base Architecture
Sprint 2	1.-8.4	Data & History	Room database, Distance/ Duration logic, History view
Sprint 3	8.-15.4	Statistics	Data processing, Statistics calculations, Statistics view, EditProfile view, Unit tests
Sprint 4	15.-22.4	UI & Navigation	Navigation, NavBar, Ui polishing, Main view, Profile view, Unit tests, Ui tests, database tests
Sprint 5	22.-29.4	Refactoring	Bug fixing, Navigation Onboarding, Ui polishing, Unit/UI tests, Architecture refactoring
Sprint 6	29.4-6.5	Refactoring & Documentation	Architecture refactoring, Ui polishing, Unit/Ui testing, Documentation

Workload was tracked with Story Points during Sprint Planning.

Work Distributions

	Farnaz	Emmi
Sprint 0	• Figma Design • Project Plan • Add Folder Structure	• App Icon • Project Plan • GitHub README
Sprint 1	• GPS Implementation	• Google Map API
Sprint 2	• History View • Filter UI	• Room Database • Filter Logic
Sprint 3	• Fix: GPS permission • Statistics View	• Database Update • EditDogProfile View • Unit Tests
Sprint 4	• Main & Profile View • Navigation • Theme • Factory	• Architecture Refactoring • Unit Tests
Sprint 5	• UI Tests • Unit Tests, • Database Tests	• Architecture Refactoring Factory • Fix Errors • DataStore (Navigation, Onboarding)
Sprint 6	• UI improvements: Dark Theme • Testing, Jacoco Report	• UI Improvements: Strings, Spacing, NavigationRail, Screen sizes, App Icon, Fix: TrackingScreen, Final Presentation, Demo Video

Jira

As a user, I want to filter history by week or day so that I can easily view walk routines

Kuvaus

We need to implement a filtering mechanism that allows users to toggle between daily/ weekly view.

Alitehtävät

100 % valmiina

Työtehtävä	Prioritee	Käsittelij	Tila
APP-74 Implement Filter UI	M..	F.	VALMIS
APP-75 Add filterstate to ViewModel	M..	F.	VALMIS
APP-76 Implement filter logic	M..	E.	VALMIS

Yhdistetyt työtehtävät

Lisää linkitetty työtehtävä

Tapahtuma

Työtila

mobile-app-team

Yhteenveto | Aikajana | Taulu | Kehitysjono | Kalenteri | Luettelo | Kehitys | Koodi | Raportit | Arkistoidut ty

Hae työtehtävää | Suodatin 1 | Ryhmitä

Työtehtävä	Käsittelijä	Raportoija	Prioriteetti	Tila	Ratkaisu
APP-71 GitHub	E...	E...	High	VALMIS	Valmis
APP-67 As a user, I want to filter history by week or day so t...	E...	E...	Medium	VALMIS	Valmis
APP-65 As a user, I want to see recent walks with distance a...	E...	Fa...	Medium	VALMIS	Valmis
APP-56 Sprint-2-Review	E...	E...	Medium	VALMIS	Valmis
APP-30 As a user I want to store my dog's previous walks, s...	E...	E...	High	VALMIS	Valmis
APP-23 As a user I want to see daily and weekly data of my ...	E...	Fa...	High	VALMIS	Valmis
APP-7 Sprint-2-Retrospective	E...	E...	Medium	VALMIS	Valmis
APP-2 Sprint-2-Planning	E...	E...	Medium	VALMIS	Valmis

Luo 8/8

Challenges

Sprint 1
- Emulator location issues (resolved with mock data)

Sprint 2
- Refined database architecture: Switched from String-converters to a separate GPS entity for better scalability.

Sprint 3
- Challenges with time management

Sprint 4
- Debugging "silent crashes" in navigation.
- Testing identified architecture that could be optimized

Sprint 5
- Version control conflict: Overwritten files led to temporary feature loss

Sprint 6
- Challenges with time management

Overall

- Time management
- Smaller group size – heavy workload

What went well

- Team Work
- Learning Process
- App works :)

Future Enhancements

More Optional Features!

- **Dog breed API:** Information about different breeds and their recommended activity levels
- **Health log:** Allows owners to record observations about the dog's droppings for health monitoring
- **Weather API:** Shows weather
- **Notifications:** Reminds the user when the dog hasn't been active enough
- **AI:** Provides simple weekly activity analysis

+ UI Improvements

Thank you.

Questions?

Wishing you all a great summer!

Links

Jira:
https://metropolia-mobile-app-project.atlassian.net/jira/software/projects/APP/boards/34?atlOrigin=eyJpIjoiYzVhZDFiZTA3ZGE2NGNiYjk2ZjhlZmY1Y2RkMTc4MzQiLCJwIjoiaiJ9

GitHub:
https://github.com/meemmi/mobile-app-project/tree/main